Steuereinheit – Befehlsübersicht

Alle Befehle, die die Control Unit (NeoTrellis 4×4) per ESP-NOW an das Grid sendet. Quellen: `controller/controller.ino`, `grid/grid.ino`, `grid/grid_config.h`.

1. Tasten der Control Unit

> **Pad physisch um 90° im Uhrzeigersinn montiert.** Die Zahlen sind die NeoTrellis-Tastennummern (Chip-Nummern); durch die Drehung liegt Taste **12** oben links, **0** oben rechts usw. Die Grafik zeigt die **echten physischen Positionen** — das, was man vor sich sieht und drückt. Im Code bleibt intern die logische Nummerierung; `PHYS2LOG` / `LOG2PHYS` in `controller.ino` gleichen die Drehung an Ein- und Ausgang aus.



Alle 16 Tasten sind belegt. Taste **4** ist jetzt **Speed-Sweep** (Animations-Geschwindigkeit fährt automatisch ein min→max→min-Dreieck), Taste **10** ist jetzt **Strobe-Speed** (globaler Strobe-Takt für alle drei Strobes). Taste **6** = **Blackout**, Taste **5** = **Modul-Spot** (alle unten beschrieben). OTA braucht keine Taste — es läuft dauerhaft ohne Trigger (Abschnitt 9).

Navigation

Taste	Funktion	Bedienung	Wirkung	LED
0	**Plus** `+`	tippen / halten (Repeat)	einen Modus weiter – oder den aktiven Parameter erhöhen	türkis
1	**Minus** `–`	tippen / halten (Repeat)	einen Modus zurück – oder den aktiven Parameter senken	türkis

Parameter-Tasten

Alle gleich bedient: **2 s halten** → Einstellmodus, danach wirken `+` / `–` auf diesen Parameter. Details in Abschnitt 3.

Taste	Parameter	LED im Standard
13	**Helligkeit**	Meter blau→rot
14	**Geschwindigkeit**	Meter blau→rot
9	**Hue-Speed**	Meter blau→rot
8	**Farbe** (Hue-Basis)	aktueller Grid-Tint
15	**Strobe-Farbe**	aktuelle Strobe-Farbe

Strobes (momentary – wirken nur, solange gehalten)

Taste	Funktion	Wirkung im Grid	LED
3	**Strobe**	Blitze in einem von 5 Looks – bei jedem Druck ein anderer; zusätzlich `+` / `−` → Pause zwischen den Blitzen regeln	aktuelle Strobe-Farbe
7	**Modus-Strobe**	blinkt die aktuelle Animation	violett
11	**Castle-Strobe**	statischer Text-Strobe „CASTLE / 2026"	blaugrün

Die 5 Strobe-Looks (Taste 3): bei **jedem Druck** wird ein anderer Look gezogen und für die Dauer des Drucks beibehalten. Die Auswahl ist zufällig, aber nie zweimal derselbe hintereinander (Shuffle-Bag: alle 5 werden gemischt durchgespielt, dann neu gemischt).

Look	Bild
Quadrate	zufällige Quadrate pro Blitz (der bisherige Look)
Voll	die ganze Wand blitzt
Streifen	6 zufällige ganze Strips pro Blitz
Ringe	quadratische Ringe, von der Mitte nach außen
Viertel	pro Blitz ein zufälliges Viertel der Wand

Farbe (Taste 15), Helligkeit und Pause gelten für alle Looks gleich; die Blitzdauer bleibt fix bei 10 ms.

Frequenz: die Periode ist **exakt** `STROBE_FLASH_MS` + Pause, also 40 … 260 ms (= **25 … 3,8 Hz**), eingestellt über 3 (Strobe) halten + `+` / `−`. Sie liegt auf einem absoluten Zeitraster (`togaStrobeOn()` in `toga_proto.h`) und ist damit **unabhängig von der Strip-Länge** – Grid und alle Module blitzen mit derselben Frequenz. Früher wurden Intervalle aneinandergehängt, wodurch die Dauer eines `FastLED.show()` (~30 µs pro LED) jeden Zyklus verlängerte und lange Strips langsamer liefen als kurze.

Was die Länge weiterhin bestimmt, ist das **Tastverhältnis**, nicht die Frequenz: ein Strip, der 15 ms zum Ausgeben braucht, kann nicht nur 10 ms leuchten. Das ist die Datenleitung, nicht der Takt.

Helligkeit: der Blitz ist auf **70 %** des Vollausschlags gedeckelt (`STROBE_MAX_BRI` = 178 von 255, in `toga_proto.h`). Das gilt für Grid **und** Module – bei den Modulen greift der Deckel nach deren Gain-Trim, ein Modul auf 200 % Gain blitzt also nicht heller als die Wand. Castle-Strobe (11) und Modus-Strobe (7) sind davon nicht betroffen.

Gleicher Takt auf jedem Modus (Fix): Alle **drei** Strobes – Strobe (3), Modus-Strobe (7) und Castle-Strobe (11) – teilen sich **eine** globale Strobe-Speed und laufen jetzt mit **voller Loop-Rate** auf einem absoluten Zeitraster. Früher lief nur die Strobe-Taste (3) so; Modus- und Castle-Strobe wurden nur so oft abgetastet, wie der gerade laufende Modus rendern konnte – auf einem schweren Modus (viel Rechenlast) also seltener, wodurch **derselbe** Strobe mal schnell, mal langsam, mal scheinbar gar nicht lief. Der **Modus-Strobe** rendert die laufende Animation außerdem nur noch **einmal pro Blitz** und hält das Bild, statt sie während des ganzen An-Fensters neu zu berechnen – das nahm dem Controller die Last und hält den Takt modusunabhängig gleich. Ergebnis: alle Strobes blitzen auf jedem Modus im **selben** Rhythmus; die Speed (Taste 10 oder eine Strobe-Taste halten + `+` / `–`) wirkt auf alle drei gleich.

Modul-Spot (Taste 5) — momentary, nur Module

Solange gehalten zeigen **alle mit der Control Unit verbundenen Module** einen einzelnen hellen **Laufpunkt**, der mit kurzem warmem Schweif über den Strip wandert (bei Panel-Modulen ein wandernder senkrechter Balken). Das **Grid bleibt unberührt** und läuft in seinem aktuellen Modus weiter. Loslassen → jedes Modul kehrt binnen ~100 ms zu seinem vorherigen Effekt zurück.

Läuft — wie die Strobes — über ein momentanes Flag (`TOGA_F_SPOT`), das auf jedem Paket inkl. Heartbeat mitreist; **kein NVS-Schreiben**. Der eigene Rendercode `renderSpot()` liegt in `module_effects.h` und ist kein `MODE_TABLE`-Eintrag. Die **Geschwindigkeit** (Taste 14) skaliert das Tempo des Punkts mit; Strobe/Castle/Modus-Strobe/Blackout haben Vorrang. Alle Module folgen unabhängig von ihrer Follow-Maske (Absicht: **alle** verbundenen Module).

> Neu geflasht werden müssen **Control Unit und alle Module**; das Grid nicht.

Auto-Modus (Taste 12)

Tippen schaltet durch vier **Stimmungen**: 1 → 2 → 3 → 4 → 1 … Jede Stimmung ist eine feste Auswahl von Modi mit einem eigenen Wechselintervall. Die Taste blinkt (~2 Hz) in der Farbe der laufenden Stimmung; ist der Automodus aus, bleibt sie dunkel.

Anzeige im Grid: Im Auto-Modus wird **keine Modus-Nummer** eingeblendet – die Wand läuft still durch. Nur beim **Weiterschalten der Stimmung mit Taste 12** blendet die Wand oben rechts **`A1` … `A4`** ein – 1 s lang, in der Eigenfarbe der Stimmung, also in derselben Farbe, in der Taste 12 gerade blinkt (Tint inklusive: die Ziffer dreht sich mit der Wand mit). Das `A` trägt dabei die Aussage, nicht die Farbe – `A2` und Modus 2 würden sich sonst nur im Farbton unterscheiden, und der Tint schiebt die Stimmungsfarben übereinander (Stimmung 2 landet bei Tint 150 auf dem Grün von Stimmung 4).

> Die automatischen **Moduswechsel** selbst blenden nichts ein (früher blitzte bei jedem Auto-Schritt die Modus-Nummer auf – das ist bewusst entfernt). Ein manueller `+` / `–`-Schritt blendet die Modus-Nummer weiterhin ein.

Beim **Ausschalten** erscheint nichts: das passiert nur durch einen manuellen `+` / `–`-Schritt, der ohnehin schon die Modus-Nummer einblendet, oder durch einen Ausfall der Steuereinheit – dort wäre ein `A0` an der Wand die Meldung eines Defekts als Einstellung.

Auto	Stimmung	Wechsel	Inhalt (aktuelle Auswahl)	LED bei Farbe 0
1	**Ruhe**	60 s	Aurora, LavaLamp, Clouds3D, SineWaves, Levitate, Heartglow, LiquidMetal, Cymatics	eisblau
2	**Space**	60 s	StarfieldWarp, Warp, StarTunnel3D, SpiralGalaxy, BlackHole2, Wormhole, Supernova, Horsehead, EarthWindow, ShootingStars, Starfield2, StarsFlying	violett
3	**Party**	60 s	BinaryRain, SquareRain, Fireworks, Firework2, Confetti, VuColumns, VuGrid, ShockwaveChain, Pulsar, SpiralEject, Earthquake, RippleRain, RainUp, Rain2	orangerot
4	**Show**	60 s	Snake, Stickman, PaddleBall, Smiley, Eye, Aliens, Tetris, BouncingLogo, TextTOGA, Radar, WarningLight	grün

Tempo: das Wechselintervall ist **fix** – standardmäßig **60 s** pro Modus, für alle vier Stimmungen gleich – und wird nur in der **Web-UI** (Modul-Manager, Intervall-Kopf) gesetzt. Es hängt **nicht** mehr an der Geschwindigkeit (Taste 14): die Speed skaliert nur die Animation selbst, nicht den Auto-Takt. Die Reihenfolge innerhalb einer Stimmung ist bei jedem Start neu gemischt (Fisher-Yates) und öffnet nie zweimal hintereinander auf demselben Modus.

LED-Farbe: die Werte in der Tabelle sind die Eigenfarbe der Stimmung, also das Bild bei Farbe = 0. Darauf kommt der Grid-Tint (Taste 8 + Hue-Speed) obendrauf – die Taste dreht sich mit der Wand mit, die vier Stimmungen bleiben dabei untereinander unterscheidbar.

Die Farbtöne stehen als `TOGA_AUTO_HUE` in `toga_proto.h` – geteilt, weil Taste 12 und die `A1` … `A4`-Anzeige der Wand dieselbe Stimmung benennen. Zwei Tabellen würden auseinanderlaufen und Taste und Wand einander widersprechen lassen. Kein Wire-Wert: ein anderer Farbton kostet **keinen** `TOGA_VERSION`-Bump. Beide Enden rechnen den Ton aber über **verschiedene** HSV-Kurven aus (Trellis: klassisches 6-Regionen-HSV, Grid: FastLEDs Rainbow) – die vier Werte sind so gewählt, dass sie auf beiden Seiten auf derselben Farbe landen. Ändert man einen, muss man beide Enden ansehen.

Ausschalten: ein manueller Moduswechsel mit `+` / `−` beendet den Automodus sofort – die Taste hört auf zu blinken. Taste 12 selbst schaltet nie aus, sie zählt nur weiter. Fällt die Steuereinheit aus (kein Heartbeat), stoppt der Automodus ebenfalls. Die automatischen Wechsel werden **nicht** gespeichert: nach einem Neustart läuft wieder der zuletzt von Hand gewählte Modus.

Sonstiges

Taste	Funktion	Bedienung	Wirkung	LED
2	**Beat-Match**	im Takt tippen (~4 Taps) · **1× tippen + 2 s warten = aus**	getapptes Tempo als **Glow-Rhythmus** (nicht Geschwindigkeit)	dunkelrot, pulsiert im erkannten Takt mit
4	**Speed-Sweep**	tippen (Umschalter)	die **Animationsgeschwindigkeit** fährt automatisch ein min→max→min-Dreieck (Periode per Web-UI); Module laufen synchron mit; erneut tippen = aus	orange, blinkt bei aktivem Sweep
6	**Blackout**	tippen (Umschalter)	Grid **und** Module werden schwarz; erneut tippen = zurück	weiß, hell wenn aktiv, sonst schwacher Marker
10	**Strobe-Speed**	**1× tippen** → Justiermodus, dann `+` / `−`	globaler **Strobe-Takt** für alle drei Strobes (3/7/11) gemeinsam; alternativ: eine Strobe-Taste halten + `+` / `−`	Meter blau→rot

Blackout (Taste 6)

Ein **Umschalter**, kein Momentary: einmal tippen → alles schwarz, nochmal → der laufende Modus ist wieder da. Der Zustand reitet – wie die Strobes – auf **jedem** Paket mit (ein verlorenes kostet ≤ 100 ms).

- **Der Modus läuft im Dunkeln weiter.** Man kann während des Blackouts blind mit `+` / `−` weiterschalten; beim Aufheben erscheint sofort der neue Modus. Die virtuelle Animationszeit läuft durch, es gibt also keinen Sprung.
- **Strobes blitzen durch.** Strobe (3), Modus-Strobe (7) und Castle-Strobe (11) liegen im Renderpfad **vor** dem Blackout und sind daher sichtbar – genau der Sinn eines Blackouts auf der Fläche.
- **Am Pad** bleiben nur Blackout (6) selbst sowie Strobe (3), Modus-Strobe (7), Castle-Strobe (11) und Auto (12) beleuchtet; alle anderen Tasten sind dunkel.
- **Sicherheitsnetz:** Blackout wird an `senderAlive` gekoppelt – fällt die Control Unit aus oder wird sie ausgeschaltet, kommt die Wand von allein zurück. Er wird **nie gespeichert**: nach einem Neustart läuft wieder der Modus, kein Schwarz.
- **Kein `TOGA_VERSION`-Bump:** Blackout nutzt ein freies Flag-Bit (`0x08`). Alte Firmware setzt es nie und ignoriert es – ein nicht neu geflashtes Modul bleibt also nur hell, statt aus dem Tritt zu geraten. Für ein wirklich komplettes Schwarz sollten Grid, Control Unit und alle Module trotzdem zusammen neu.

2. Befehle an das Grid

Befehl	Ausgelöst durch	Mitgeschickt	Wirkung im Grid
Heartbeat	automatisch alle 100 ms + sofort bei jeder Strobe-Flanke, bei Taste 6 (Blackout) und bei Taste 12 (Auto)	die drei Strobe-Zustände, der **Blackout** und der Automodus	hält die Verbindung; bleibt er länger als 400 ms aus, gilt der Sender als weg
Modus-Schritt	`+` / `−` im Standard	Richtung vorwärts / rückwärts	Modus wechseln (läuft über alle 90 Modi um), gespeicherte Helligkeit des Modus laden, Modus-Nummer oben rechts einblenden
Parameter-Schritt	`+` / `−` im Einstellmodus, oder Strobe halten + `+` / `−`	welcher Parameter, Richtung	ändert genau diesen Parameter – siehe Parameter-Tabelle
Beat-Tap	jeder Druck auf Taste 2, sofort	nichts – nur „jetzt"	ein einzelner Tap; **das Grid** erkennt daraus Tempo und Phase und legt den **Glow-Rhythmus** darauf

Zusätzlich meldet **jede** Nachricht – auch der Heartbeat – den aktiven Automodus (0 = aus, 1–4), den **Blackout** (an/aus) und die drei Strobe-Zustände mit:

Zustand	Taste	Wirkung
Strobe	3	weiße/farbige Blitze, solange gehalten – der Look wird im Grid bei jeder steigenden Flanke neu gezogen
Modus-Strobe	7	blinkt die aktuelle Animation
Castle-Strobe	11	statischer Text-Strobe „CASTLE / 2026"
Blackout	6	Umschalter: Grid und Module schwarz; Strobes bleiben sichtbar, Modus läuft im Dunkeln weiter

3. Parameter

Parameter	Taste	Schritt	Bereich	Verhalten am Ende
Helligkeit	13	±5	0 … 255	begrenzt (0 = komplett aus, 255 = volle Helligkeit), pro Modus gespeichert
Geschwindigkeit	14	±0.1	0.1 … 4.0	begrenzt; regelt zusätzlich das Wechseltempo des Automodus
Farbe (Hue-Basis)	8	±6	ganzer Farbkreis	läuft um
Strobe-Farbe	15	±6	ganzer Farbkreis	läuft um
Hue-Speed (Auto-Rotation)	9	±1 Stufe (≈ ±8 % Tempo)	0 … 100	begrenzt
Strobe-Pause	3 (Strobe) halten + `+` / `−`	±5 ms	30 … 250 ms	begrenzt

Jede Parameteränderung wird im Grid persistent gespeichert.

Hue-Speed (Taste 9) ist die einzige **nicht lineare** Skala: ein Druck ändert das Rotationstempo um ca. 8 % – über den ganzen Regelbereich gleich fein. Die Skala reicht von „Farbkreis in ~10 Minuten" (Stufe 1) bis „Farbkreis in ~0,3 s" (Stufe 100), das Maximum ist also unverändert.

Stufe	Farbkreis dauert
0	aus, keine Rotation
1	~10 min
25	~1,5 min
50	~15 s
75	~2 s
100	~0,3 s

Vorher war die Skala linear 0…40 und lag damit **komplett** zwischen 13 s und 0,3 s pro Umlauf: schon die kleinste Stufe hat die Wand rotieren lassen, ein langsames Driften war gar nicht einstellbar. Ein alter gespeicherter Wert (0…40) wird auf der neuen Skala **langsamer** – einmal neu einstellen.

4. Bedienmodell (Zustände der Control Unit)

Zustand	Eintritt	`+` / `–` wirken auf	Austritt
Standard	Start	**Modus**	–
Einstellmodus	Parameter-Taste 2 s halten	**den aktiven Parameter**	dieselbe Taste erneut tippen (kein Timeout)

Beat-Match ist **kein Zustand mehr**: Taste 2 ist immer tap-bereit und ändert nichts an `+` / `–`. Der einzige Timeout dort ist der Ausschalter: 1 Tap + 2 s Stille beendet den Glow (Abschnitt 4a).

Solange **Taste 3 (Strobe) gehalten** wird, wirken `+` / `–` immer auf die Strobe-Pause – unabhängig vom Zustand.

Im Einstellmodus blinkt die aktive Parameter-Taste (~4 Hz) in ihrer eigenen Farbe, alle anderen Tasten sind dunkel – außer `+` / `–`, (beim Einstellen der Strobe-Farbe) Taste 3 (Strobe) als Live-Vorschau und Taste 12 (Auto), solange der Automodus läuft: Parameter einstellen oder Beat tappen beendet ihn nicht.

Auto-Repeat von `+` / `–` im Einstellmodus: nach 500 ms Halten alle 80 ms, ab 2 s alle 40 ms, ab 4 s alle 16 ms. Beim Regeln der Strobe-Pause: nach 400 ms alle 60 ms.

4a. Beat-Match (Taste 2)

Einfach im Takt tippen – kein Scharfschalten. Die Taste selbst pulsiert im erkannten Takt mit: das ist die Rückmeldung, dass das Grid gelockt hat.

Ausschalten: einmal tippen und warten. Ein **einzelner** Tap, gefolgt von **2 s ohne weiteren Tap**, schaltet den Glow ab – die Taste hört auf zu pulsieren und leuchtet wieder nur dunkelrot. Ab **zwei** Taps ist es immer ein Tempo, nie ein Ausschalten; die beiden Gesten können sich also nicht verwechseln.

Der Preis dafür, bewusst bezahlt: ein einzelner Tap **korrigiert die Phase nicht mehr**, er beendet den Glow. Nachziehen geht mit zwei Taps (das Grid liest das als passendes Intervall und zieht die Phase ohnehin nach).

Wer rechnet: die Taste schickt jeden Tap **einzeln und sofort** ans Grid, und das Grid macht die ganze Erkennung (`grid/grid_beat.h`). Das ist der Kern der Sache: der Tap ist der **Phasen-Anker** für den Helligkeits-Peak, und der ist nur auf dem Knoten etwas wert, der die Frames zeichnet.

Einlernen	Mittel der letzten 4 Intervalle; nach ~4 Taps ist das Tempo gelockt
Gültig	200 … 2000 ms Tap-Abstand (30 … 300 BPM); Ausreißer werden ignoriert, ohne ein laufendes Tempo zu verwerfen
Prellen	Taps < 100 ms nach dem letzten zählen nicht
Stabilität	Annahme: das Tempo ändert sich im Lied nicht. Passende Intervalle (±15 %) justieren die Periode nur um 5 %; doppelte Intervalle gelten als verpasster Beat. Erst **6** unpassende Intervalle in Folge verwerfen das Tempo (Meldung auf Serial)
Phase	der Tap ist der Anker. Beim ersten Lock hart, danach ziehen neue Taps die Phase nur zu 30 % nach – sonst würde der Puls um ein Tempo zappeln, das er längst kennt
Verlauf	**Sägezahn**: schlagartig 100 % exakt auf dem Beat, dann linear auf `BEAT_MIN` = 25 % bis zum nächsten. Bewusst kein Sinus – der steigt *vor* dem Beat wieder an und wirkt dadurch verfrüht
Vorlauf	`BEAT_LATENCY_MS` = 70 ms: der Peak wird um diesen Wert vorverlegt (Vorhersage über die gelockte Periode). Wirkt der Blitz zu spät → erhöhen, zu früh → senken
Laufzeit	endlos im erkannten Tempo – bis neue Taps ein neues setzen oder ein einzelner Tap ihn beendet
Aus	1 Tap + 2 s Stille (`BEAT_SOLO_MS`). Wird wie jede Tempoänderung gespeichert: nach einem Neustart bleibt der Glow aus. Randfall: bei ≤ 30 BPM ist der Tap-Abstand selbst 2 s – dort schaltet der erste Tap kurz ab, die folgenden Taps lernen das Tempo neu ein

Der Puls moduliert **nur die globale Helligkeit** über *alle* Animationen (Faktor 0.25 … 1.0 auf alle Kanäle) und ersetzt keinen Modus. Die Animationsgeschwindigkeit bleibt **unberührt**. Das Tempo überlebt einen Neustart (10 s nach dem letzten Tap gespeichert – pro Tap zu schreiben würde den Flash verschleißen). Jeder Tap meldet auf Serial (115200) Intervall und BPM.

Die Module bekommen Tempo *und* Phase über den 200-ms-Sync und pulsieren dieselbe Kurve – die Latenz-Kompensation ist in der gesendeten Phase schon drin.

Strobe (3): der Blitz ist fix und so kurz wie möglich (`STROBE_FLASH_MS` = 10 ms – kürzer geht nicht, ein LED-Refresh über alle 12 Strips dauert ~10–15 ms). Nur die Pause ist regelbar.

5. Rückmeldung vom Grid (Sync-Broadcast)

Das Grid sendet alle 200 ms seinen **absoluten** Zustand als Broadcast. Zwei Empfänger nutzen ihn: die Control Unit für ihre Tasten-LEDs, und die Module als ihre einzige Zustandsquelle (siehe Abschnitt 6).

Gemeldet wird	**wofür**
Helligkeit, Geschwindigkeit, aktueller Modus	Meter-Anzeige auf den Parameter-Tasten
Farbe (Basis **und** Auto-Rotation), Hue-Speed, Strobe-Farbton	Farb-Parameter
exakte aktuelle Tint-Farbe	Taste 8 (Farbe) und der Blitz des Modus-Strobes
exakte aktuelle Strobe-Farbe	Tasten 15 (Strobe-Farbe) und 3 (Strobe)
Strobe-Pause, Beat-Periode und Beat-Phase	damit die Module im selben Takt und phasengleich pulsieren

6. Module (weitere ESP32-LED-Einheiten)

Neben dem Grid hören beliebig viele **Module** mit — eigenständige ESP32-Boards mit eigenen LEDs (`module/module.ino`).

Wie ein Modul gesteuert wird

Quelle	Was das Modul davon nimmt
Control-Unit-Broadcast (100 ms)	**nur** die drei Strobe-Zustände + Lebenszeichen — latenzarm, weil Blitze sofort sitzen müssen
Grid-Sync-Broadcast (200 ms)	**alles andere**: Helligkeit, Farbe, Hue-Speed, Geschwindigkeit, Strobe-Farbe und -Pause, Beat

Das Modul rechnet **keine** `+` / `−`-Schritte selbst nach, sondern spiegelt den absoluten Grid-Zustand. Damit rastet ein Modul, das später bootet oder Pakete verpasst, binnen ~200 ms wieder exakt auf das Grid ein und kann nie auseinanderlaufen.

Unterschiede zum Grid

- **Modus-Schritte werden ignoriert.** Ein Modul spiegelt den absoluten Grid-Modus 1:1 („folgt Grid"), oder es sitzt auf einem festen Modus / eigenen Effekt (siehe Eigenständiger Betrieb).
- **Castle-Strobe** wird zum Vollflächen-Blitz in der Tint-Farbe — ein Modul hat keinen Font für „CASTLE / 2026" und oft nur 8 Zeilen.
- **Keine AGC.** Stattdessen ein einmaliger `gain`-Trim pro Modul (die AGC des Grids ist auf dessen Buffer mit den toten Spacer-LEDs kalibriert und würde auf einem voll leuchtenden Panel eine *andere* Helligkeit ergeben).

Konfiguration — überall dieselbe Firmware, pro Gerät einmalig über die serielle Konsole (115200), Werte liegen im NVS:

Befehl	Bedeutung
`show` / `help`	Zustand, MAC, Gruppe, Geometrie, was gerade vom Grid kommt
`group <0–2>`	0 = alle · 1 = Grid · 2 = Module (Standard)
`w <n>` / `h <n>`	Spalten / Zeilen · `h 1` = einfacher Streifen
`leds <n>`	Kurzform für `w n` + `h 1`
`pin <n>`	Daten-GPIO: 4, 5, 16, 17, 18 oder 21
`effect <0–67>`	Modul auf **einen** Grid-Modus festnageln · `effect off` = aus · `effect –1` / `follow` = dem Grid-Modus folgen (Standard)
`local <0–9>`	**eigenständiger** Effekt (kein Grid nötig): 0 Solid · 1 Atmen · 2 Puls · 3 Komet · 4 Funkeln · 5 Feuer · 6 Regenbogen · 7 Regenbogen-Fluss · 8 Regenbogen-Plasma · 9 Regenbogen-Funkeln
`auto <0–2>`	eigenständiges **Tanz-Programm**: 0 Mix · 1 Farbe · 2 Regenbogen (wechselt die Effekte selbst durch)
`color <0–255>`	**Farbe** für den eigenständigen Betrieb: 0 = nativ (kein Tint), 1…255 = Farbton. Wirkt nur, wenn das Modul **nicht** „folgt Grid" ist
`gain <10–200>`	Helligkeits-Trim in %, gegen das laufende Grid einstellen
`bmon`	Li-Ion-Unterspannungsschutz — **fest an, nicht schaltbar** (der Befehl meldet das nur). Das Modul misst alle 10 s die Zellspannung (GPIO34); nach 3× < Cutoff (3,30 V) → Deep-Sleep, LED-Pin hochohmig, Aufwachen nur per echtem Power-Cycle. **Plausibilitäts-Untergrenze 2,50 V:** liegt der Messwert darunter, ist keine Zelle am Messpfad (Akku-Schalter aus, Betrieb über Ladebuchse) → wird ignoriert, **kein** Deep-Sleep. `battVolt` steht in der Web-UI und in `show`.
`save` / `reboot`	`effect` / `local` / `auto` / `color` / `gain` wirken sofort; `w` / `h` / `pin` erst nach `save` + `reboot`

Die blaue Onboard-LED zeigt die Verbindung: leuchtet = Control Unit lebt.

Eigenständiger Betrieb (Farbe · Effekt · Auto-Tanz)

Ein Modul, das **nicht** an die Show gebunden ist — als einzelner Streifen an der Wand, auf dem Boden, an der Bar — soll für sich alleine gut aussehen. Sobald ein Modul **nicht** auf „folgt Grid" steht, ist es **eigenständig**: es nimmt seine Farbe und seinen Effekt nicht mehr vom Grid, sondern aus eigener Einstellung. Zurück auf „folgt Grid" → es spiegelt wieder exakt die Wand.

Farbe. Eine feste Farbe (`color <0–255>` bzw. die Farbfelder in der Web-UI). Die eigenständigen Effekte werden in genau dieser Farbe gemalt; `0` = nativ (kein Tint). Bei den Regenbogen-Effekten ist die Farbe egal — die drehen den ganzen Farbkreis selbst.

Effekte — schlicht und gut aussehend auf **einem** Streifen, nicht die 2D-Zitate des Grids:

Gruppe	Effekte
feste Farbe, pulsiert & fadet	Solid · Atmen · Puls · Komet · Funkeln · Feuer
Regenbogen	Regenbogen (ganzer Streifen zyklisch) · Regenbogen-Fluss · Regenbogen-Plasma · Regenbogen-Funkeln

Auto-Tanz. Statt einen Effekt fest zu wählen, lässt ein Auto-Programm das Modul selbst durch eine Auswahl wechseln (~14 s, mit der Geschwindigkeit skaliert) — zum Tanzen, ohne dass jemand am Handy sitzt:

Programm	Inhalt
Mix	Farbfades **und** Regenbogen abwechselnd
Farbe	nur die Fades in der festen Farbe (Atmen, Puls, Komet, Funkeln)
Regenbogen	nur die Regenbogen-Effekte

Alles per Serial (`local` / `auto` / `color`) **oder** in der Web-UI: pro Modul ein gruppiertes **Effekt**-Menü (Auto-Tanz · Standalone · folgt Grid · Grid-Modi) und eine **Farb**-Reihe. Die Auswahl liegt im NVS und übersteht einen Neustart. Ein Modul ohne Controller (Notmodus, siehe Abschnitt 9) serviert dieselbe Seite selbst — so lässt sich ein einzelner Streifen komplett ohne Grid einrichten.

Effekt und Gain gehen auch per Weboberfläche — siehe Abschnitt 8. Über Web gesetzt werden beide sofort **gespeichert** (kein `save` nötig): die Konsole hat man vor der Nase und kann korrigieren, das Handy nicht — ein Wert, der beim nächsten Einschalten still zurückspringt, wäre dort ein Fehlerbericht.

8. Weboberfläche (Module einzeln ansteuern)

Die Control Unit liefert die Web-UI selbst aus — erreichbar unter `toga-ctl.local` im gleichen AP, den sie ohnehin aufmacht. Kein eigenes Board mehr (die frühere `webserver/` -Variante ist entfernt, sie ist in die Control Unit gefaltet). Die Seite treibt keine LEDs und steuert die Show nicht — sie hört nur zu und konfiguriert Module.

Jedes Modul meldet sich **1×/s** von selbst (`TogaHelloMsg`); die Seite listet alles, was sie hört, mit MAC, Geometrie, Pin, Gruppe, laufendem Effekt, Gain, Uptime und Controller-Verbindung. Pro Modul einstellbar: **Effekt** (Grid-Modus, eigenständiger Effekt, Auto-Tanz oder „folgt Grid"), **Farbe** (für den eigenständigen Betrieb) und **Gain**. Gesendet wird `TogaModCfgMsg` , adressiert an **eine MAC** — jedes andere Modul verwirft das Paket.

Module haben vorher nie gesendet. Das Hello ist ein **Bericht, nie ein Befehl**: ein Modul wendet weiterhin keine eigenen Schritte an und spiegelt den Grid-Zustand vollständig. Geht ein Hello verloren, veraltet nur die Liste.

7. Protokoll & Build

Alle vier Sketches teilen sich **ein** Wire-Format: `libraries/TogaProto/src/toga_proto.h` (Magic, Version, Gruppen-ID, Sequenznummer, Structs, gemeinsame Konstanten). Die Structs werden nicht mehr von Hand dupliziert.

- **Alles läuft per Broadcast** auf einem festen Kanal (`TOGA_CHANNEL` = 1). Kein Knoten darf sich je bei einem **Router** einloggen — das würde ihn auf dessen Kanal zwingen und die ESP-NOW-Verbindung still killen. Beim AP der Control Unit ist das anders: der steht selbst auf `TOGA_CHANNEL` , also kostet die Anmeldung niemanden seinen Platz auf dem Bus (Abschnitt 9).
- **Broadcast wird nicht quittiert und nicht wiederholt.** Einmalige Befehle (Modus, Parameter, Beat) gehen deshalb 3× raus mit *einer* Sequenznummer; die

Empfänger führen den ersten aus und verwerfen die Wiederholungen. Der Heartbeat braucht das nicht — er kommt 10×/s ohnehin wieder.

- **Gruppen-ID** steckt im Paket (0 = alle); die Control Unit sendet immer 0. Gezieltes Ansteuern **einzelner** Module läuft nicht über Gruppen, sondern über die MAC im `TogaModCfgMsg` der Weboberfläche (Abschnitt 8).

Voraussetzung zum Bauen: `libraries/TogaProto` muss in den Arduino-Libraries verlinkt sein — einmalig:

```
ln -s "$PWD/libraries/TogaProto" ~/Documents/Arduino/libraries/TogaProto
arduino-cli lib list | grep -i toga     # muss TogaProto zeigen
```

Gepinnt und nicht verhandelbar: **FastLED 3.6.0**, **esp32-core 2.0.17**, Partition **min_spiffs**. FastLED 3.10.x bricht ESP-NOW, core 3.x ändert die Empfangs-Callback-Signatur — beides ist mit `#error` abgesichert.

Warum `min_spiffs` und nicht mehr `huge_app`: OTA braucht **zwei** App-Partitionen, zwischen denen umgeschaltet wird. `huge_app` hat nur eine (`app0`, 3 MB) — damit ist OTA schlicht unmöglich, egal wie man es anstellt. `min_spiffs` hat `app0` + `app1` à 1,875 MB; der größte Sketch (Grid) füllt davon 46 %. SPIFFS schrumpft dabei, was nichts kostet: kein Sketch benutzt es.

```
FQBN="esp32:esp32:esp32:PartitionScheme=min_spiffs"
arduino-cli compile --fqbn "$FQBN" --libraries ./libraries --upload -p <port> grid/
```

Wird das Wire-Format geändert (`TOGA_VERSION`, aktuell **4** — v4 brachte Batterie-Telemetrie im Hello + `battMon`-Toggle im ModCfg), müssen **Grid, Control Unit und alle Module zusammen** neu geflasht werden. Ein Knoten auf einer alten Version ist für alle anderen still unsichtbar.

9. OTA — Firmware über die Luft

Nach **einem** letzten USB-Durchgang (siehe `min_spiffs` oben) läuft jedes Update über WLAN — **jederzeit, ohne Trigger, ohne dass ein Knoten die Show verlässt**.

Warum das geht: die **Control Unit macht den AccessPoint auf** — SSID `togalights`, Passwort `andmagic`, fest auf **Kanal 1**. Und Kanal 1 ist `TOGA_CHANNEL`, also genau der Kanal, auf dem der ESP-NOW-Bus ohnehin läuft. Ein ESP32 hat nur ein Radio: sich bei einem *Router* anzumelden würde es auf dessen Kanal zerren und den Knoten still vom Bus nehmen. Bei **unserem** AP passiert das nicht — er steht schon auf dem richtigen Kanal. Jeder Knoten ist deshalb gleichzeitig im Netz **und** auf dem Bus.

Zwei Nebeneffekte, beide gut: die Anlage braucht **überhaupt keinen Router** mehr, und jedes Gerät ist unter seinem eigenen Namen erreichbar.

Knoten	Hostname
Grid	`toga-grid.local`
Control Unit	`toga-ctl.local`
Modul	`toga-mod-<letzte 3 MAC-Bytes>.local` (z.B. `toga-mod-7d75c0.local`)

Flashen (der Mac muss dafür in `togalights` sein — das Netz hat kein Internet, das ist Absicht):

```
./flash-ota.sh module toga-mod-7d75c0.local
./flash-ota.sh grid   toga-grid.local
./flash-ota.sh controller toga-ctl.local
```

macOS fragt beim ersten Mal, ob `python3` eingehende Verbindungen annehmen darf — das muss erlaubt werden, sonst hängt der Upload bei „Sending invitation" (das ESP verbindet sich zurück zum Rechner).

Der NVS bleibt bei alldem erhalten — er liegt in beiden Partitionsschemata an derselben Adresse, und der Upload schreibt sie nicht an. Aber: **niemals** `esptool erase_flash`, das nimmt ihn mit.

Sicherheitsnetz: Grid, Control Unit und Module halten ein frisches Image zuerst auf Probe (`verifyRollbackLater`) und segnen es erst ab, wenn sie sich bewährt haben — 60 s Betrieb **plus** ein Paket vom Bus, spätestens aber nach 5 min. Stürzt das neue Image vorher ab oder bootloopt es, holt der Bootloader von allein das alte zurück. Keine Leiter. **Grenze:** das fängt Abstürze, keine Geschmacksfragen — ein Image, das sauber bootet und Unsinn rendert, segnet sich selbst ab. Deshalb archiviert `flash-ota.sh` jede geflashte Binary.

Wenn die Control Unit aus ist, gibt es keinen AP und damit kein OTA. Der Bus läuft trotzdem weiter (alle Knoten sind auf Kanal 1 gepinnt, dafür braucht es keine Anmeldung) — nur flashen kann man dann nicht.